

A practical and flexible PUF-based end-to-end anonymous authentication protocol for IoT

Sensen Li^{*}, Yicai Huang, Bin Yu

Zhengzhou Information Science and Technology Institute, Zhengzhou 450001, China

ARTICLE INFO

Keywords:

Internet of Things (IoT)
End-to-end anonymous authentication
Key agreement
Physical Unclonable Function (PUF)

ABSTRACT

With the explosive development of the Internet of Things (IoT), its security has become a critical concern. As a lightweight hardware primitive, Physical Unclonable Function (PUF) provides a promising solution for IoT security. Nevertheless, as per the knowledge of the authors, most of the existing PUF-based anonymous authentication protocols for IoT are not suitable for end-to-end IoT applications due to their flexibility or security. Specifically, there are two main issues with existing related protocols: (1) the end-to-end anonymous authentication between IoT devices requires the participation of a trusted third party, which seriously affects the flexibility of interaction; (2) most related protocols provide weak anonymity, that is, they are anonymous against only eavesdroppers. To solve these problems, a new PUF-based end-to-end anonymous authentication protocol is proposed. Without needing the real-time participation of the third party, the proposed protocol realizes end-to-end direct mutual authentication and session key agreement while maintaining strong anonymity. It also provides an online dynamic updating mechanism for security parameters. The security analysis shows that the proposed protocol has perfect forward secrecy while resisting various known attacks including physical attacks and modeling attacks. Meanwhile, it outperforms related protocols in terms of protocol rounds and communication costs.

1. Introduction

With the rapid development of mobile communication and wireless sensor network technology, the Internet of Things (IoT) has been applied to almost all aspects of life. The Juniper research [1] pointed out that the number of IoT devices will rapidly increase to 83 billion by 2024. The application of IoT devices in healthcare, finance, transportation, and other fields has brought about great convenience to our daily life. However, it also poses a threat to personal privacy and property security [2]. Therefore, privacy protection and identity authentication are crucial in IoT security.

Anonymous authentication can realize legitimacy authentication between entities without disclosing their identity information. To improve IoT security, many authentication protocols have been proposed. Hammi et al. [3] and Tedeschi et al. [4] proposed certificateless authentication protocols for IoT. Though these protocols are lightweight and low-cost, they do not consider the anonymity of users' identity information. In 2019, Malani et al. [5] designed an anonymous device access control scheme for IoT, but this scheme requires the distribution

and management of certificates, which may impose a huge burden on the server. Without using certificates, Cao et al. [6] exploited the timestamp mechanism to design an anonymous authentication protocol. And Srinivas et al. [7] proposed an efficient authentication scheme based on Schnorr's signature. Considering the limited capabilities of IoT devices, Ding et al. [8] proposed a lightweight authentication protocol based on elliptic curve cryptography and signcryption techniques. In addition, several studies [9,10] proposed anonymous authentication schemes for IoT considering multiple factors.

All the anonymous authentication schemes mentioned above are based on traditional symmetric or asymmetric cryptography, which assumes that the shared key or the private key is securely stored locally in the devices. However, sensor devices in IoT usually have poor self-protection ability, and they are generally deployed in unattended scenes. Adversaries may obtain the security parameters stored in sensor devices by physical attacks [11], which will invalidate the schemes above. Besides, anonymous authentication schemes based on traditional cryptography cannot effectively solve the problem that malicious users harm the interests of service providers by copying and sharing the

^{*} Corresponding author.

E-mail address: lss589@163.com (S. Li).

<https://doi.org/10.1016/j.comnet.2024.110426>

Received 28 August 2023; Received in revised form 24 March 2024; Accepted 15 April 2024

Available online 27 April 2024

1389-1286/© 2024 Elsevier B.V. All rights reserved.

authentication credentials [12].

With the characteristics of unpredictability, stability, and uniqueness, Physical Unclonable Function (PUF) [13] provides a feasible way to resist physical attacks by exploiting the hardware features [14]. In recent years, many scholars have proposed authentication protocols for IoT by using PUF, such as [14–24].

1.1. Challenges and motivation

In PUF-based authentication protocols, challenge-response pairs (CRPs) of PUF are usually utilized to achieve authentication and key establishment, which involves sharing CRPs during the registration phase and using these authentication factors during the authentication phase. Some protocols directly store the raw CRPs, which may be subject to modeling attacks [25] implemented by adversaries on the premise of collecting a subset of CRPs of the PUF. As a result, the PUF-based protocol should adopt appropriate strategies to avoid storing raw CRPs. Besides, most protocols rely on the trusted third party to perform the interaction process of anonymous authentication. For end-to-end IoT applications, these protocols undoubtedly have poor efficiency and flexibility. In addition, due to the fact that IoT devices typically communicate in open channels and some applications, such as smart healthcare and e-commerce, involve user privacy, the PUF-based authentication protocol needs to resist various attacks including eavesdropping attacks, replay attacks, data tampering attacks, etc., while maintaining strong anonymity for IoT devices.

According to the analysis above, the challenges of designing PUF-based end-to-end anonymous authentication protocol for IoT mainly include two aspects, flexibility and security. Specifically, the challenge in terms of flexibility refers to realizing end-to-end direct authentication without relying on the trusted third party. Besides, the security challenge is to ensure strong anonymity while resisting various attacks. As per the knowledge of the authors, existing PUF-based anonymous authentication protocols for IoT cannot simultaneously overcome the aforementioned challenges. The reasons for these difficulties are as follows. The CRPs used as authentication factors in PUF-based protocols are typically bound to the specific device identity. However, the requirement for strong anonymity poses obstacles to verifying the legitimacy of these authentication factors. Although relying on the trusted third party can solve this problem, it comes at the cost of compromising flexibility. Therefore, designing an end-to-end authentication protocol that considers security as well as flexibility remains challenging.

1.2. Research contributions

Motivated by the above-mentioned challenges, we propose a new PUF-based protocol to achieve flexible end-to-end authentication with strong anonymity for IoT. The key contributions are as follows.

- (1) The end-to-end anonymous authentication does not need the real-time participation of the third-party server and only requires “three handshakes”, which is more practical and flexible than existing related protocols.
- (2) The proposed protocol meets the requirement of strong anonymity. During the process of anonymous authentication, no one, including eavesdroppers, verifiers, or even the server responsible for security parameter distribution and updating, can obtain the real identity of IoT terminal devices that act as the protocol initiators.
- (3) Using the lightweight hardware security primitive PUF, the proposed protocol does not store any secret parameters in resource-constrained IoT devices, which effectively resists physical attacks while providing perfect forward secrecy. Besides, without needing to store the raw CRPs of PUF, the proposed protocol is free from modeling attacks.

- (4) By adopting certificateless public key cryptography, the proposed protocol averts the management of digital certificates while settling the key escrow issue.
- (5) The utilization of device hardware features enables the proposed PUF-based protocol to prevent malicious users from copying and sharing the authentication credentials, thus protecting the interests of service providers effectively.

1.3. Organization

The rest of the paper is organized as follows. In Section II, we review the related work. Section III provides the preliminaries. Section IV discusses the system models. In Section V, the proposed PUF-based anonymous authentication protocol is introduced. The security analysis and performance analysis of the proposed protocol are provided in Section VI and Section VII, respectively. Finally, this paper is concluded in Section VIII.

2. Related work

Over the past years, many scholars have conducted research on PUF-based anonymous authentication for IoT.

To address the anonymous authentication between the terminal device and the server, several lightweight schemes [14–16] have been proposed by combining PUF with symmetric cryptography, which can effectively achieve mutual authentication between the IoT device and server while protecting the privacy of IoT devices with low resource overhead. But these schemes mainly focus on the secure communication between the terminal device and the server, which do not involve end-to-end authentication for IoT devices.

Aiming at the end-to-end anonymous authentication for IoT, researchers have presented some interesting PUF-based schemes. In 2019, Gope et al. [17] proposed a physically secure anonymous mutual authentication protocol for industrial wireless sensor networks using lightweight cryptographic primitives, such as hash function, PUF, and XOR. However, this protocol cannot support the noisy PUF environment, for the reason that environmental factors may result in one or several of the output bits of PUF being incorrect for any challenge [15]. Subramani et al. [18] used PUF and fuzzy extractor to design their scheme, which provided an efficient anonymous authentication solution for WBAN. But the adoption of bilinear pairing operations with high computational complexity makes this scheme unsuitable for resource-limited IoT devices. Suganthi et al. [19] proposed an efficient end-to-end mutual authentication scheme for the IoT-based healthcare environment, which can ensure the security and privacy of the patient with low resource overhead. But in this scheme, the raw CRPs of the patient's PUF are directly stored in the gateway, which may lead to modeling attacks including machine learning attacks if the gateway becomes malicious [12]. Similarly, the schemes [20,21] also face the threat of modeling attacks. Chatterjee et al. [22] developed an authentication and key exchange protocol for IoT by combining PUF with identity-based encryption without needing to store CRPs explicitly. This protocol could resist the modeling attacks, but bilinear pairing operations are adopted. Besides, the aforementioned PUF-based authentication schemes [17–22] all require the participation of a trusted third party, such as the server or gateway, during the mutual authentication of two IoT devices, which makes their flexibility relatively poor and significantly affects the efficiency. To solve this problem, Li et al. [23] and Zheng et al. [24] respectively proposed efficient end-to-end mutual authentication protocols for IoT without needing the participation of a trusted third party. However, these protocols [23,24] fail to assure the anonymity of IoT devices. In addition, the end-to-end anonymous authentication protocols [17–21] for IoT all assume weaker anonymity and these protocols are anonymous against only eavesdroppers. In other words, the verifier can obtain the real identity of IoT devices, which does not align with the basic notion of anonymous authentication [12].

In summary, most of the existing PUF-based anonymous authentication protocols for IoT are not suitable for end-to-end IoT applications due to their security or flexibility. To solve these issues, we concentrate on designing a more practical and flexible end-to-end anonymous authentication protocol for IoT.

The comparison of properties between the proposed protocol and relevant protocols is shown in Table 1. The properties of the proposed protocol outperform these of relevant protocols. Without needing the real-time participation of the trusted third party during the anonymous authentication process, while ensuring meeting the security requirements, the proposed protocol is more suitable for IoT applications.

3. Preliminaries

In this section, some background information involved in the proposed protocol is briefly introduced, including the relevant mathematical assumptions, PUF, and fuzzy extractor.

3.1. Relevant mathematical assumptions

Let $e_{a,b}$ be the secure elliptic curve selected from the finite field F_q , where q is a large prime. $P \in e_{a,b}(F_q)$ is a generator of the cyclic group G , whose order is a large prime p . The definitions of the elliptic curve discrete logarithm (ECDL) assumption and the elliptic curve computational Diffie-Hellman (ECCDH) assumption are as follows.

Definition 1 The ECDL assumption. Given $P, xP \in G$, it is intractable for a probabilistic polynomial time (PPT) algorithm $\mathcal{S}$ to calculate the unknown parameter $x \in \mathbb{Z}_p^*$. This assumption is denoted as $\Pr[\mathcal{S}(P, xP) = x | x \in \mathbb{Z}_p^*] \leq \epsilon_{ECDL}$, where ϵ_{ECDL} is negligible.

Definition 2 The ECCDH assumption. Given $P, xP, yP \in G$, it is intractable for a PPT algorithm $\mathcal{S}$ to calculate xyP , where $x, y \in \mathbb{Z}_p^*$ are unknown parameters. This assumption is denoted as $\Pr[\mathcal{S}(P, xP, yP) = xyP | x, y \in \mathbb{Z}_p^*] \leq \epsilon_{ECCDH}$, where ϵ_{ECCDH} is negligible.

3.2. PUF

PUF is constructed by exploiting the unique variability in the integrated circuit manufacturing process. Once the manufacturing process is completed, PUF will integrate with the device hardware and obtain a unique mapping relationship.

Definition 3 PUF model [23]. $PUF_D : \{0, 1\}^{l_{in}} \rightarrow \{0, 1\}^{l_{out}}$ bound to device D is a mapping from challenge $c \in \{0, 1\}^{l_{in}}$ to response $r \in \{0, 1\}^{l_{out}}$ and has the following properties.

- (1) *High efficiency.* PUF_D is easy to implement, and its evaluation is efficient.
- (2) *Unpredictability.* For a PPT adversary $\mathcal{A}$, the probability to distinguish $PUF_D(c_0)$ from random $r_0 \leftarrow_R \{0, 1\}^{l_{out}}$ is no more than

Table 1
Comparison of properties.

Protocols	P1	P2	P3	P4	P5	P6
Gope et al. (2019) [17]	×	×	✓	×	×	weak
Suganthi et al. (2019) [19]	✓	×	✓	×	×	weak
Banerjee et al. (2019) [20]	✓	×	✓	×	×	weak
Chatterjee et al. (2019) [22]	✓	✓	×	✓	×	×
Liu et al. (2020) [21]	✓	×	✓	×	×	weak
Li et al. (2021) [23]	✓	✓	✓	✓	✓	×
Subramani et al. (2022) [18]	✓	×	×	×	×	weak
Zheng et al. (2022) [24]	✓	✓	✓	✓	✓	×
Proposed Protocol	✓	✓	✓	✓	✓	strong

Note: P1: Supporting the noisy PUF environment, P2: Storing no raw CRPs, P3: Using no complex operations, P4: Free from modeling attacks, P5: Without needing trusted third party, P6: Anonymity.

ϵ_{puf} , even if $\mathcal{A}$ has obtained the response set $\mathcal{R}$ corresponding to part of the challenge set $\mathcal{C}(c_0 \notin \mathcal{C})$, where ϵ_{puf} is a negligible value. This property shows that it is difficult to predict the responses of PUF.

- (3) *Stability.* The maximum distance between responses generated by PUF_D to the same challenge is tolerable, which can be eliminated by the fuzzy extractor to be described in the next section. This property is denoted as $\Pr[\text{dist}(y, z) > \epsilon | x \leftarrow_R \{0, 1\}^{l_{in}}, y \leftarrow PUF_D(x), z \leftarrow PUF_D(x)] \leq \epsilon_s$, where dist represents the distance, ϵ is a tolerable value, and ϵ_s is negligible.
- (4) *Uniqueness.* For any two PUFs, PUF_D and $PUF_{D'}$, even though their production process and circuit composition are the same, the distance between their responses to the same challenge is no less than ϵ' . This property is denoted as $\Pr[\text{dist}(y, z) < \epsilon' | x \leftarrow_R \{0, 1\}^{l_{in}}, y \leftarrow PUF_D(x), z \leftarrow PUF_{D'}(x)] \leq \epsilon_U$, where ϵ' is a tolerable value, and ϵ_U is negligible.

3.3. Fuzzy extractor

The operating environment may affect the stability of PUF. To enable PUFs to generate stable responses, fuzzy extractor and PUF are usually used in combination.

Definition 4 Fuzzy Extractor [26]. A typical fuzzy extractor includes two processes:

- (1) *Gen* : $\{0, 1\}^n \times \{0, 1\}^l \leftarrow \{0, 1\}^n$. For the input sequence $\omega \in \{0, 1\}^n$, this process produces the output $r \in \{0, 1\}^n$ and the help data $p \in \{0, 1\}^l$, which is denoted as $(r, p) \leftarrow \text{Gen}(\omega)$.
- (2) *Rep* : $\{0, 1\}^n \leftarrow \{0, 1\}^n \times \{0, 1\}^l$. This process can reproduce the output r from the sequence $\bar{\omega}$ and the corresponding help data p , where $\bar{\omega}$ could be any sequence close to ω . This process can be denoted as $r \leftarrow \text{Rep}(\bar{\omega}, p)$, if $\text{dist}(\omega, \bar{\omega}) \leq \epsilon$ and $(r, p) \leftarrow \text{Gen}(\omega)$, where ϵ is a tolerable value.

4. System models

4.1. Typical application scenarios and network model

Authentication plays an important role in achieving secure communication for IoT. In some scenarios with high security requirements, protecting the identity privacy of terminal devices and users is also crucial. Taking the smart healthcare and the intelligent transportation as examples, the typical application scenarios of anonymous authentication are introduced as follows.

- (1) The smart healthcare. Terminal devices aggregate the vital sign data of users, such as blood pressure, heartbeat, pulse, etc., and send them to remote medical monitoring service nodes owned by medical staff. Terminal devices hope to send data to legitimate service nodes without exposing their identity information in order to protect user privacy.
- (2) The intelligent transportation. Vehicle mounted terminal devices obtain their location information by sending requests to location service nodes. Terminal devices hope to confirm the legitimacy of location service nodes while protecting identity privacy, and service nodes also need to authenticate terminal devices.

This paper adopts the network model shown in Fig. 1. The entities mainly include the management server, terminal devices, and service nodes. The management server is a trusted entity, which is responsible for network configuration, device registration, and security parameter distribution and updating.

Due to the limited resources and poor self-protection abilities, this paper adopts the PUF-enabled devices to resist physical attacks.

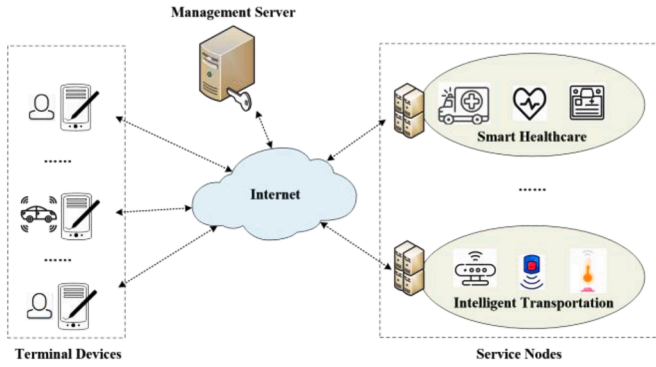


Fig. 1. Network model.

According to the properties of PUF, the following reasonable assumptions are made.

- (1) PUF is embedded with the microcontroller of the specific device. The communication between them is secure and cannot be accessed from outside [14].
- (2) Any attempt to tamper/separate the PUF from the device will make the PUF useless [14].

4.2. Threat model

The communication among terminal devices, service nodes, and the management server in IoT is implemented based on the Internet, which is open and public. As a result, adversaries may perform malicious attacks on the communication process of IoT devices. The Dolev-Yao (DY) model [27] points out that the adversary against the cryptographic protocol could entirely control the public channel, thus obtaining the messages through eavesdropping, and even modifying, forging, or deleting messages.

Considering the poor self-protection abilities of terminal devices and service nodes, this paper further extends the DY threat model by increasing the physical attack capability of the adversary. Specifically, the adversary considered in this paper has the following capabilities.

- (1) Physical attack capability. The adversary can physically capture resource-constrained devices to obtain all the stored parameters.
- (2) Passive attack capability. The adversary can eavesdrop and analyze the messages transmitted in the public channel.
- (3) Active attack capability. The adversary can interfere with the communication or destroy the authentication process by modifying, forging, or deleting the transmitted messages.

4.3. Security model

According to the capabilities of the adversary considered in this paper, the security model proposed in [28] is optimized. Let $\mathcal{P}$ denote the proposed authentication protocol and $\mathcal{D}_x$ denotes the x^{th} session instance of the protocol entity $\mathcal{D}_x$.

Definition 5 The formal description of the adversary's attack capabilities. The attack behaviors of the PPT adversary $\mathcal{A}$ can be characterized by oracle queries, and the specific definitions are as follows.

- (1) $\text{Corrupt}(\mathcal{D}_x)$: In this query, $\mathcal{A}$ can obtain all the parameters stored by $\mathcal{D}_x$. This query is used to simulate the physical attack capability of $\mathcal{A}$.
- (2) $\text{Execute}(\mathcal{P})$: In this query, $\mathcal{A}$ can obtain all the parameters in the channel according to the protocol $\mathcal{P}$. This query is used to simulate the passive attack capability of $\mathcal{A}$.

- (3) $\text{Send}(\mathcal{D}_x, m)$: $\mathcal{A}$ constructs the challenge and send it to $\mathcal{D}_x$. Then, $\mathcal{A}$ obtains the response generated by $\mathcal{D}_x$ according to the protocol $\mathcal{P}$. This query is used to simulate the active attack capability of $\mathcal{A}$.
- (4) $\text{Test}(\mathcal{P})$: This query does not simulate the actual attack capability of $\mathcal{A}$. Instead, it is used to characterize the semantic security of the session key negotiated by protocol entities. When $\mathcal{A}$ performs this query, a coin is randomly flipped. If the coin is facing up, the session key will be returned to $\mathcal{A}$; otherwise, a value of the same length will be randomly selected and returned. This query can be performed only once by $\mathcal{A}$.

Definition 6 The semantic security of the session key. If $\mathcal{A}$ can correctly distinguish whether the return value obtained in $\text{Test}(\mathcal{P})$ is the real session key, it is said that $\mathcal{A}$ wins the coin game, and this event is denoted as $\text{Succ}(\mathcal{A})$. $\mathcal{A}$'s advantage can be formally defined as $\text{Adv}_{\mathcal{P}}^{\text{SK}}(\mathcal{A}) = 2\text{Pr}[\text{Succ}(\mathcal{A})] - 1$. If $\text{Adv}_{\mathcal{P}}^{\text{SK}}(\mathcal{A})$ is negligible, the protocol $\mathcal{P}$ is session key (SK)-secure.

5. Proposed protocol

The main phases of the proposed protocol include: (1) system initialization, (2) service node registration, (3) terminal device registration, (4) anonymous authentication, (5) service node parameter update, (6) terminal device parameter update. The registration phase is implemented in a secure manner as in protocols [22,23], and it is made up of service node registration and terminal device registration. Besides, the update phase includes the update of service node parameters and terminal device parameters, which can be dynamically implemented online.

The process of the proposed protocol is shown in Fig. 2. In the *System Initialization* phase, the management server MS selects system parameters, and this step is executed only one time when the network is established. In the *Service Node Registration* phase, the service node obtains its security parameters from MS , and this step is executed when new service node joins the network. Similarly, the terminal device undergoes the step of *Terminal Device Registration* to obtain its security parameters from MS when joining the network or needing to request services from a new service node. The *anonymous authentication* phase is the core of the protocol, which is executed when terminal devices request services from service nodes, enabling mutual authentication and session key establishment. In addition, the service nodes and terminal devices can update their security parameters in the *Service Node Parameter Update* phase and *Terminal Device Parameter Update* phase, respectively. Table 2 lists the key notations used in the protocol.

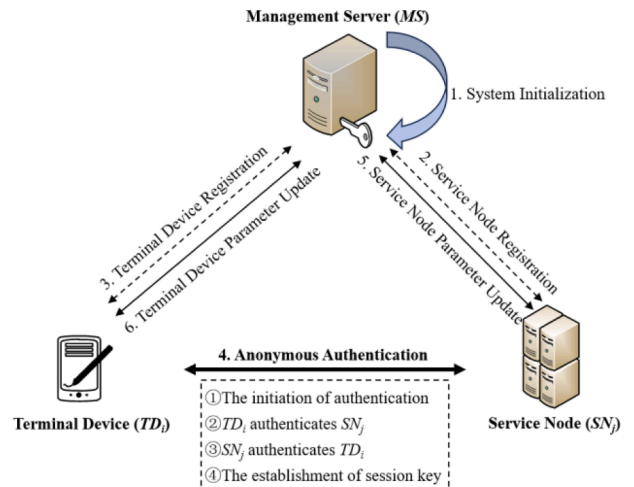


Fig. 2. The process of the proposed protocol.

Table 2

Notations.

Notation	Description
MS	the management server in IoT
SN_j	the service node in IoT
TD_i	the terminal device in IoT
(P_{pub}, k_M)	the public/private key pair of MS
$H_i(\cdot) (i = 1, 2, 3, 4, 5)$	the one-way hash functions
(X_j, x_j)	the public/private key pair of SN_j
$\langle (Y_{i1}, y_{i1}), (Y_{i2}, y_{i2}) \rangle$	the public/private key pair of TD_i
$(\bar{y}_i, \bar{Y}_i)$	the temporary public/private key pair of TD_i
$PUF_i(\cdot), PUF_j(\cdot)$	the PUF of TD_i and SN_j
$Gen(\cdot), Rep(\cdot)$	the generation and reproduction functions of fuzzy extractor
c	the challenge of PUF
r	the response of PUF
p	the help data generated by fuzzy extractor
ts_j	the timestamp selected by SN_j
n, e, u, v, nc	the random values
k_{ij}, k_{ji}	the session key computed by TD_i and SN_j

5.1. System initialization

The management server MS first selects a secure elliptic curve $E_{a,b}$: $y^2 = x^3 + ax + b$ from the finite field F_q , where q is a large prime, and a, b satisfy $4a^3 + 27b^2 \neq 0 \pmod q$. Also, it chooses the cyclic group G , whose order is a large prime p and the generator is $P \in E_{a,b}$. Then, MS randomly selects its private key $k_M \in Z_p^*$ and calculates $P_{pub} = k_MP$. Subsequently, MS chooses five secure one-way hash functions, $H_1 : \{0, 1\}^* \rightarrow Z_p^*, H_2 : \{0, 1\}^* \times G \rightarrow Z_p^*, H_3 : \{0, 1\}^* \times G \times G \rightarrow Z_p^*, H_4 : \{0, 1\}^* \times G \times G \times G \rightarrow Z_p^*, H_5 : \{0, 1\}^* \times G \times G \times G \times G \rightarrow Z_p^*$. Finally, MS publishes the public parameters $\{E_{a,b}, q, p, P, P_{pub}, H_1, H_2, H_3, H_4, H_5\}$ and secretly keeps its private key k_M .

5.2. Service node registration

Here, the service node SN_j is taken as an example to describe the service node registration phase, which can be divided into three steps.

- (1) SN_j sends its identity information $\langle SN_j \rangle$ to the management server MS as the registration request.
- (2) On receiving the registration request, MS randomly chooses $x_j \in Z_p^*$ and calculates $X_j = x_jP$. Then, MS sends the message $\langle x_j, X_j \rangle$ to SN_j as the response.
- (3) SN_j randomly chooses $c_j \in Z_p^*$, and uses its PUF to obtain (r_j, p_j) with the assistance of the fuzzy extractor by executions $\omega_j \leftarrow PUF_j(c_j)$ and $(r_j, p_j) \leftarrow Gen(\omega_j)$. Then, SN_j selects the current timestamp ts_j as its registration time, calculates $px_j = r_j \oplus x_j$, $hr_j = H_1(r_j || c_j || |x_j| || ts_j)$, and sends the message $\langle c_j, p_j, hr_j, ts_j \rangle$ to MS .

Finally, SN_j stores the parameters $\langle px_j, X_j, ts_j \rangle$ and MS stores $\langle SN_j, c_j, hr_j, p_j, x_j, X_j, ts_j \rangle$. Other parameters generated in the registration phase are all deleted.

5.3. Terminal device registration

Taking the terminal device TD_i as an example, the operations of the terminal device registration phase are as follows.

- (1) The terminal device TD_i sends the message $\langle TD_i, SN_j \rangle$ to the management server MS , where SN_j is the identity of the target service node.
- (2) On receiving the message, MS queries the parameters $\langle SN_j, c_j, hr_j, p_j, x_j, X_j, ts_j \rangle$ corresponding to SN_j from its database and randomly selects $n_{i1}, n_{i2} \in Z_p^*$. Then, MS calculates $Y_{i1} = n_{i1}P$, $Y_{i2} = n_{i2}P$, y_{i1}

$= k_M + n_{i1} \cdot hr_j + x_j, y_{i2} = k_M + n_{i2} \cdot hr_j + x_j$, respectively, and sends the message $\langle (y_{i1}, Y_{i1}), (y_{i2}, Y_{i2}), X_j, c_j, p_j, ts_j \rangle$ to TD_i .

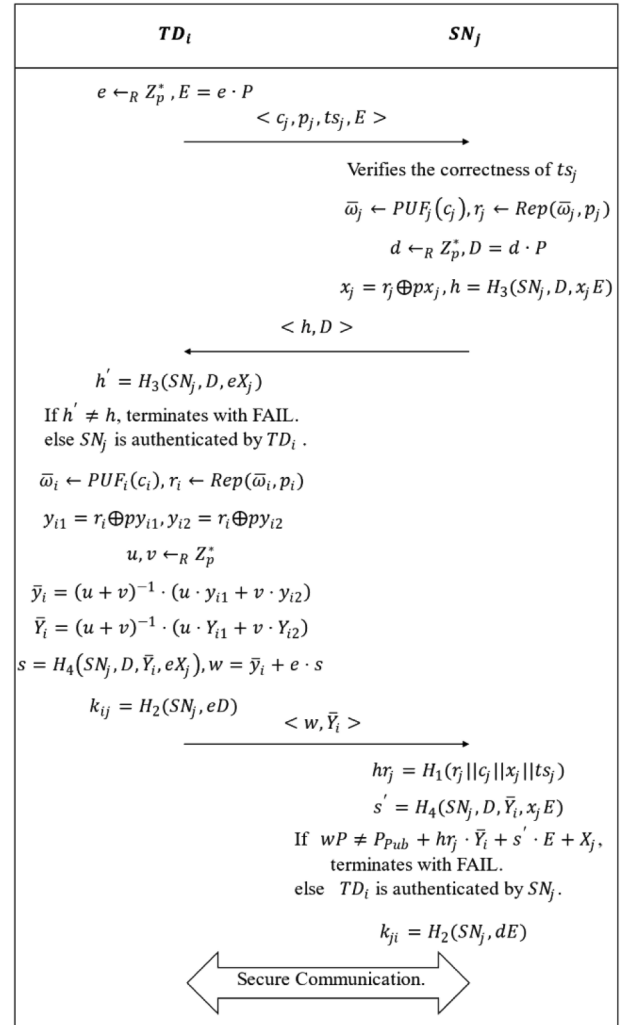
- (3) TD_i randomly selects $c_i \in Z_p^*$, and uses its PUF to obtain (r_i, p_i) with the assistance of the fuzzy extractor by executions $\omega_i \leftarrow PUF_i(c_i)$ and $(r_i, p_i) \leftarrow Gen(\omega_i)$. Then, TD_i calculates $py_{i1} = r_i \oplus y_{i1}$, $py_{i2} = r_i \oplus y_{i2}$.

Finally, TD_i stores the parameters $\langle SN_j, c_j, p_j, X_j, ts_j, c_i, p_i, (py_{i1}, Y_{i1}), (py_{i2}, Y_{i2}) \rangle$ and MS stores $\langle TD_i, SN_j, y_{i1}, y_{i2}, Y_{i1}, Y_{i2} \rangle$. And they both delete other parameters generated in this phase.

5.4. Anonymous authentication

After registration, terminal devices can request relevant services from the target service nodes, which verify the legitimacy of terminal devices through the anonymous authentication process. Considering the situation where terminal device TD_i wants to obtain services from service node SN_j , the process of anonymous authentication is described below as shown in Fig. 3.

- (1) First, TD_i reads the parameters $\langle c_j, p_j, X_j, ts_j, c_i, p_i, (py_{i1}, Y_{i1}), (py_{i2}, Y_{i2}) \rangle$ corresponding to SN_j from the device storage. Then, TD_i randomly selects $e \in Z_p^*$ and calculates $E = eP$. Subsequently, TD_i sends the message $\langle c_j, p_j, ts_j, E \rangle$ to SN_j as the anonymous authentication request.

**Fig. 3.** Anonymous authentication.

- (2) On receiving the request, SN_j first verifies the correctness of the received registration time ts_j . If so, SN_j uses its PUF to obtain r_j with the assistance of the fuzzy extractor by executions $\bar{w}_j \leftarrow PUF_j(c_j)$ and $r_j \leftarrow Rep(\bar{w}_j, p_j)$. Then, SN_j randomly selects $d \in Z_p^*$, calculates $D = dP$, $x_j = r_j \oplus px_j$, $h = H_3(SN_j, D, x_jE)$, and sends the message $\langle h, D \rangle$ to TD_i as the response.
- (3) TD_i calculates $h' = H_3(SN_j, D, eX_j)$ and checks whether the equation $h' = h$ holds. If that is the case, TD_i authenticates SN_j and continues the following procedures; otherwise, the authentication fails and TD_i terminates the communication.
 TD_i obtains r_i by executions $\bar{w}_i \leftarrow PUF_i(c_i)$ and $r_i \leftarrow Rep(\bar{w}_i, p_i)$. Then, TD_i calculates $y_{i1} = r_i \oplus py_{i1}$ and $y_{i2} = r_i \oplus py_{i2}$, and randomly selects $u, v \in Z_p^*$ to obtain its temporary public/private key pair $(\bar{y}_i, \bar{Y}_i)$ by performing $\bar{y}_i = (u + v)^{-1} \cdot (u \cdot y_{i1} + v \cdot y_{i2})$ and $\bar{Y}_i = (u + v)^{-1} \cdot (uY_{i1} + vY_{i2})$. Subsequently, TD_i calculates $s = H_4(SN_j, D, \bar{Y}_i, eX_j)$, $w = \bar{y}_i + e \cdot s$, and the session key $k_{ij} = H_2(SN_j, eD)$, then sends the message $\langle w, \bar{Y}_i \rangle$ to SN_j .
- (4) SN_j calculates $hr_j = H_1(r_j || c_j || x_j || ts_j)$, $s' = H_4(SN_j, D, \bar{Y}_i, x_jE)$ and checks whether the equation $wP = P_{pub} + hr_j \cdot \bar{Y}_i + s' \cdot E + X_j$ holds. If the verification fails, SN_j terminates the communication; otherwise, SN_j authenticates TD_i and the session key $k_{ji} = H_2(SN_j, dE)$.

Due to $k_{ji} = H_2(SN_j, dE) = H_2(SN_j, edP) = H_2(SN_j, dE) = k_{ij}$, TD_i and SN_j have established the shared session key. Based on this, they can realize secure communication. After the communication is completed, TD_i and SN_j delete all the parameters generated during the anonymous authentication phase.

5.5. Service node parameter update

According to the service duration or frequency, network status, and other information, the management server MS and service nodes can dynamically update relevant parameters. Taking the service node SN_j as an example, Fig. 4 shows the operations of service node parameter update, which can be implemented online.

- (1) SN_j sends its identity information $\langle SN_j \rangle$ to MS as the update request.
- (2) On receiving the request, MS reads the parameters $\langle c_j, hr_j, p_j, x_j, X_j, ts_j \rangle$ corresponding to SN_j from the device storage and randomly selects $x'_j \in Z_p^*$. Then, MS calculates $X'_j = x'_jP$, $h_j = H_3(c_j, hr_j, p_j, x_j, x'_j, X_j, X'_j)$, and sends the message $\langle c_j, p_j, x_j \oplus x'_j, h_j \rangle$ to SN_j .
- (3) SN_j obtains r_j by executions $\bar{w}_j \leftarrow PUF_j(c_j)$ and $r_j \leftarrow Rep(\bar{w}_j, p_j)$, and calculates $x_j = r_j \oplus px_j$, $hr_j = H_1(r_j || c_j || x_j || ts_j)$, $x'_j = x_j \oplus (x_j \oplus x'_j)$, $X'_j = x'_jP$, respectively. Then, SN_j checks the validity of h_j . If the verification fails, SN_j terminates the communication; otherwise, SN_j authenticates MS and continues the following procedures.
 SN_j randomly selects $c'_j \in Z_p^*$, and obtains (r'_j, p'_j) by executions $w'_j \leftarrow PUF_j(c'_j)$ and $(r'_j, p'_j) \leftarrow Gen(w'_j)$. Then, SN_j selects the current timestamp ts'_j as its new registration time, calculates $px'_j = r'_j \oplus x'_j$, $hr'_j = H_1(r'_j || c'_j || x'_j || ts'_j)$, $h'_j = H_1(c'_j || hr'_j || hr_j || p'_j || x_j || x'_j || ts'_j)$, and sends the message $\langle c'_j, hr_j \oplus hr'_j, p'_j, h'_j \rangle$ to MS .
- (4) MS calculates $hr_j = hr_j \oplus (hr_j \oplus hr'_j)$ and checks the validity of h'_j . If the verification fails, MS terminates the communication; otherwise, MS authenticates SN_j .

Finally, SN_j stores the new parameters $\langle px'_j, X'_j, ts'_j \rangle$ and adds the old parameters $\langle px_j, X_j, ts_j \rangle$ to the revoked parameter list. MS stores the new

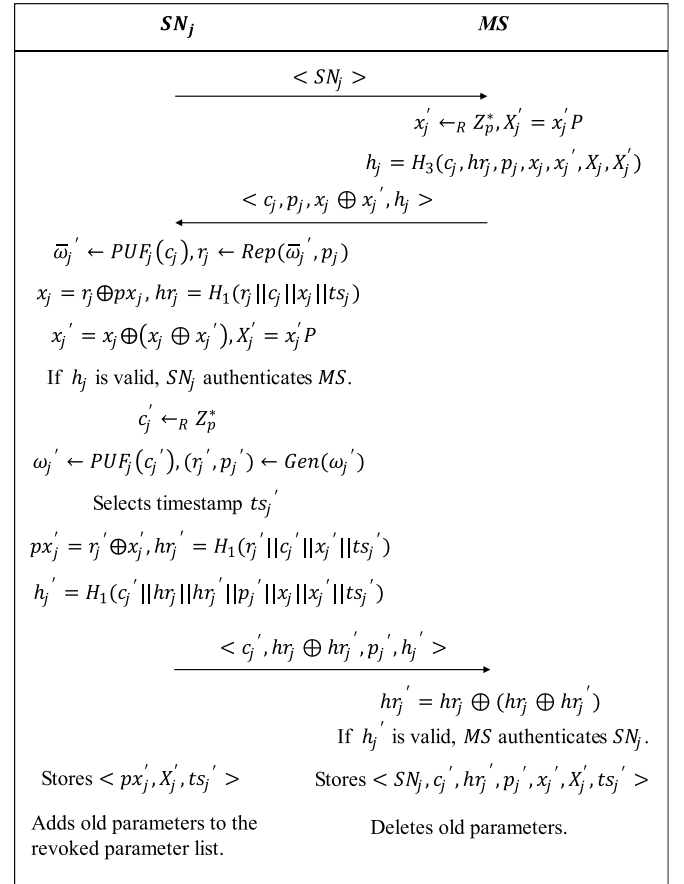


Fig. 4. Service node parameter update.

parameters $\langle SN_j, c_j, hr_j, p_j, x_j, X_j, ts_j \rangle$ and deletes the old parameters.

5.6. Terminal device parameter update

The terminal device parameter update phase is triggered during the anonymous authentication process. Taking the terminal device TD_i with old parameters requesting service from SN_j with updated parameters as an example, before the update, the parameters owned by TD_i are old $\langle SN_j, c_j, p_j, X_j, ts_j, c_i, p_i, (py_{i1}, Y_{i1}), (py_{i2}, Y_{i2}) \rangle$, while SN_j owns the updated parameters $\langle px'_j, X'_j, ts'_j \rangle$. Meanwhile, the parameters owned by MS are $\langle TD_i, SN_j, y_{i1}, y_{i2}, Y_{i1}, Y_{i2} \rangle$ and $\langle SN_j, c'_j, hr'_j, p'_j, x'_j, X'_j, ts'_j \rangle$. Obviously, TD_i owing the old parameters cannot be authenticated by SN_j , but SN_j can know TD_i is a legal terminal device that has not completed parameter update by further matching the revoked parameter list. Then, SN_j notifies TD_i to request new parameters from the management server MS . The terminal device parameter update process between TD_i and MS is described below as shown in Fig. 5.

- (1) TD_i reads the parameters $\langle SN_j, c_j, p_j, X_j, ts_j, c_i, p_i, (py_{i1}, Y_{i1}), (py_{i2}, Y_{i2}) \rangle$ from the device storage and obtains r_i by executions $\bar{w}_i \leftarrow PUF_i(c_i)$ and $r_i \leftarrow Rep(\bar{w}_i, p_i)$. Then, TD_i randomly selects $nc_1 \in Z_p^*$ and calculates $y_{i1} = r_i \oplus py_{i1}$, $y_{i2} = r_i \oplus py_{i2}$, $NC_1 = nc_1 \cdot P$, $ey_{i1} = y_{i1} \oplus H_2(nc_1 \cdot P_{pub})$. Subsequently, TD_i sends the message $\langle SN_j, ey_{i1}, NC_1 \rangle$ to MS as the update request.
- (2) On receiving the message, MS can obtain y_{i1} by performing $y_{i1} = ey_{i1} \oplus H_2(k_M \cdot NC_1)$ for the reason that $k_M \cdot NC_1 = k_M \cdot nc_1 \cdot P = nc_1 \cdot P_{pub}$. Then, using the parameters $\langle SN_j, y_{i1} \rangle$, MS can get the identity of TD_i and related parameters $\langle TD_i, SN_j, y_{i1}, y_{i2}, Y_{i1}, Y_{i2} \rangle$ by matching the parameter list. On this basis, MS reads the

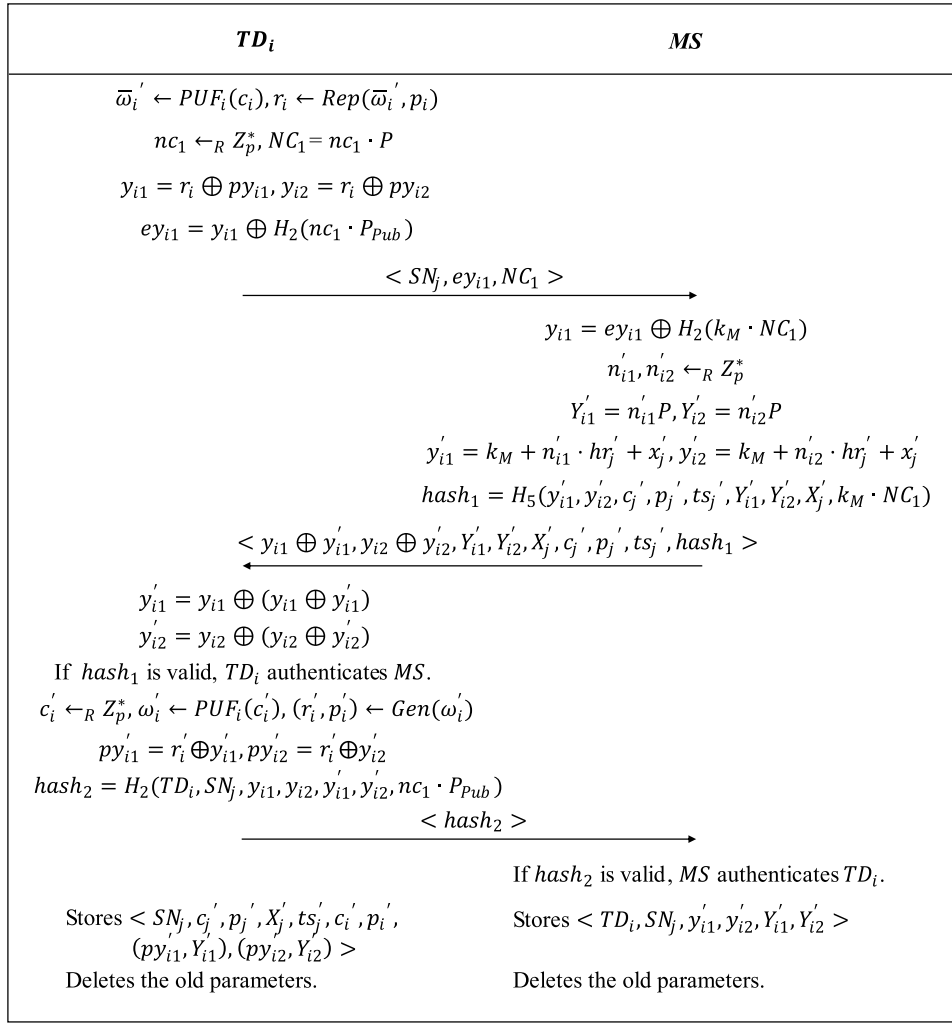


Fig. 5. Terminal device parameter update.

parameters $\langle SN_j, c_j', hr_j', p_j', x_j', X_j', ts_j' \rangle$ corresponding to SN_j , randomly selects $n'_{i1}, n'_{i2} \in Z_p^*$, and calculates $Y'_{i1} = n'_{i1}P, Y'_{i2} = n'_{i2}P, y'_{i1} = k_M + n'_{i1} \cdot hr_j' + x_j', y'_{i2} = k_M + n'_{i2} \cdot hr_j' + x_j', hash_1 = H_5(y'_{i1}, y'_{i2}, c_j', p_j', ts_j', Y'_{i1}, Y'_{i2}, X_j', k_M \cdot NC_1)$, respectively. Then, MS sends the message $\langle y_{i1} \oplus y'_{i1}, y_{i2} \oplus y'_{i2}, Y'_{i1}, Y'_{i2}, X_j', c_j', p_j', ts_j', hash_1 \rangle$ to TD_i .

- (3) TD_i calculates $y'_{i1} = y_{i1} \oplus (y_{i1} \oplus y'_{i1}), y'_{i2} = y_{i2} \oplus (y_{i2} \oplus y'_{i2})$, and checks the validity of $hash_1$. If the verification fails, TD_i terminates the communication; otherwise, TD_i authenticates MS and continues the following procedures.

TD_i randomly selects $c_i' \in Z_p^*$, and uses its PUF to obtain (r_i', p_i') with the assistance of the fuzzy extractor by executions $\omega_i' \leftarrow PUF_i(c_i')$ and $(r_i', p_i') \leftarrow Gen(\omega_i')$. Then, TD_i calculates $py'_{i1} = r_i' \oplus y'_{i1}, py'_{i2} = r_i' \oplus y'_{i2}, hash_2 = H_2(TD_i, SN_j, y_{i1}, y_{i2}, y'_{i1}, y'_{i2}, nc_1 \cdot P_{Pub})$, and sends $\langle hash_2 \rangle$ to MS .

- (4) MS checks whether the equation $hash_2 = H_2(TD_i, SN_j, y_{i1}, y_{i2}, y'_{i1}, y'_{i2}, k_M \cdot NC_1)$ holds. If the verification fails, MS terminates the communication; otherwise, MS authenticates TD_i .

Finally, MS stores the new parameters $\langle TD_i, SN_j, y'_{i1}, y'_{i2}, Y'_{i1}, Y'_{i2} \rangle$ and deletes the old parameters $\langle TD_i, SN_j, y_{i1}, y_{i2}, Y_{i1}, Y_{i2} \rangle$. TD_i stores the new parameters $\langle SN_j, c_j', p_j', X_j', ts_j', c_i', p_i', (py'_{i1}, Y'_{i1}), (py'_{i2}, Y'_{i2}) \rangle$ and deletes

the old parameters $\langle SN_j, c_j, p_j, X_j, ts_j, c_i, p_i, (py_{i1}, Y_{i1}), (py_{i2}, Y_{i2}) \rangle$. On this basis, TD_i can use the new parameters to request service from SN_j .

6. Security analysis

In this section, it is formally proved that the proposed anonymous authentication protocol is SK-secure under the random oracle model according to the *Security Model* in Section IV. Meanwhile, other security features of the protocol are analyzed by using the informal method.

6.1. Formal security analysis

Theorem 1. Let $\mathcal{A}$ be a PPT adversary against the proposed protocol $\mathcal{P}$ with q_h Hash-queries, q_e Execute-queries, and q_s Send-queries in time bound t . For the protocol participants TD_i and SN_j , the i^{th} instance of TD_i and the j^{th} instance of SN_j are denoted as TD_i^{i1} and SN_j^{j2} , respectively. Then,

$$Adv_{\mathcal{A}}^{SK}(\mathcal{A}) \leq \frac{q_h^2 + (q_s + q_e)^2 + 4q_s}{p} + 2\epsilon_{puf} + 4\epsilon_{ECDL} + 2\epsilon_{ECCDH}$$

where p is the large prime.

Proof. As in [29], a sequence of games, $Game_i (i = 0, 1, 2, 3, 4, 5, 6)$, are constructed to describe the attack behavior of $\mathcal{A}$. At the end of each game $Game_i$, $\mathcal{A}$ performs the *Test*-query. Let $Succ_i(\mathcal{A})$ denote the event that $\mathcal{A}$ wins the coin game in $Game_i$, and the probability is $\Pr[Succ_i(\mathcal{A})]$.

Game₀: This game simulates the actual attack performed by $\mathcal{A}$ on the protocol $\mathcal{P}$. According to Definition 6, we have

$$\text{Adv}_{\mathcal{P}}^{\text{SK}}(\mathcal{A}) = 2\Pr[\text{Succ}_0(\mathcal{A})] - 1.$$

Game₁: Different from **Game₀**, this game replaces the actual output of PUF with a random value of the same length. Specifically, a list $LP_{\mathcal{P}}$ is maintained for simulation $\text{PUF}_{\mathcal{P}}$ to store the record $(\text{Pin}_{\mathcal{P}}, \text{Pout}_{\mathcal{P}})$, where $\text{Pin}_{\mathcal{P}}$ is the input and $\text{Pout}_{\mathcal{P}}$ is the corresponding output of $\text{PUF}_{\mathcal{P}}$. For the input query $\text{Pin}_{\mathcal{P}}$ on $\text{PUF}_{\mathcal{P}}$, if the record $(\text{Pin}_{\mathcal{P}}, \text{Pout}_{\mathcal{P}})$ exists in $LP_{\mathcal{P}}$, $\text{Pout}_{\mathcal{P}}$ is returned; otherwise, a new random value $\text{Pout}_{\mathcal{P}}$ with the same length is selected and returned. Then, $(\text{Pin}_{\mathcal{P}}, \text{Pout}_{\mathcal{P}})$ is added to $LP_{\mathcal{P}}$. Obviously, if the adversary $\mathcal{A}$ can distinguish **Game₁** from **Game₀**, then $\mathcal{A}$ can distinguish the output of PUF and the random value. According to the *Unpredictability* of PUF in Definition 3, the probability is no more than ϵ_{puf} . Thus,

$$|\Pr[\text{Succ}_1(\mathcal{A})] - \Pr[\text{Succ}_0(\mathcal{A})]| \leq \epsilon_{\text{puf}}.$$

Game₂: In this game, the *Hash*, *Corrupt*, *Execute* and *Send* queries are simulated as follows.

- (1) *Hash* queries. A list LH_i is maintained for simulation $\mathcal{H}_i$ to store the record $(\text{Hin}_i, \text{Hout}_i)$, where Hin_i is the input and Hout_i is the corresponding output. For the hash query Hin_i , if the record $(\text{Hin}_i, \text{Hout}_i)$ exists in LH_i , Hout_i is returned; otherwise, a new random value Hout_i of the same length is selected and returned, and then $(\text{Hin}_i, \text{Hout}_i)$ is added to LH_i .
- (2) *Corrupt* queries. The *Corrupt* queries include two types, *Corrupt*(TD_i^1) and *Corrupt*(SN_j^2).
 - a) *Corrupt*(TD_i^1): For this query, the parameters $\langle \text{SN}_j, c_j, p_j, X_j, \text{ts}_j, c_i, p_i, (\text{py}_{i1}, Y_{i1}), (\text{py}_{i2}, Y_{i2}) \rangle$ stored by TD_i are returned.
 - b) *Corrupt*(SN_j^2): For this query, the parameters $\langle \text{px}_j, X_j, \text{ts}_j \rangle$ stored by SN_j are returned.
- (3) *Execute* query. For this query, all the messages transmitted in the public channel when the protocol participants *honestly* execute the proposed protocol are returned. That is, by performing the *Execute* query, $\mathcal{A}$ can obtain the parameters $\{ \langle c_j, p_j, \text{ts}_j, E \rangle, \langle h, D \rangle, \langle w, \bar{Y}_i \rangle \}$.
- (4) *Send* queries. When the *Send* queries are asked, **Game₂** first obtains the parameters $\langle \text{SN}_j, c_j, p_j, X_j, \text{ts}_j, c_i, p_i, (\text{py}_{i1}, Y_{i1}), (\text{py}_{i2}, Y_{i2}) \rangle$ and $\langle \text{px}_j, X_j, \text{ts}_j \rangle$ from the queries *Corrupt*(TD_i^1) and *Corrupt*(SN_j^2). Then, the queries are simulated as follows.
 - (a) *Send*($\text{TD}_i^1, \text{start}$): **Game₂** randomly selects $e \in Z_p^*$, calculates $E = eP$, and returns the message $\langle c_j, p_j, \text{ts}_j, E \rangle$.
 - (b) *Send*($\text{SN}_j^2, \langle c_j, p_j, \text{ts}_j, E \rangle$): **Game₂** first verifies the correctness of ts_j . If so, **Game₂** randomly selects $d \in Z_p^*$, calculates the parameters $\{D, x_j, h\}$ according to the protocol and returns the message $\langle h, D \rangle$; otherwise, **Game₂** outputs a failure and terminates the process.
 - (c) *Send*($\text{TD}_i^1, \langle h, D \rangle$): **Game₂** calculates the relevant parameters $\{h\}$ according to the protocol, and checks whether the equation $h' = h$ holds. If it holds, **Game₂** continues the following procedures; otherwise, **Game₂** outputs a failure and terminates the process. **Game₂** randomly selects $u, v \in Z_p^*$, calculates the relevant parameters $\{\bar{y}_i, \bar{Y}_i, s, w, k_{ij}\}$ according to the protocol, and then returns the message $\langle w, \bar{Y}_i \rangle$.
 - (d) *Send*($\text{SN}_j^2, \langle w, \bar{Y}_i \rangle$): For this query, **Game₂** calculates the relevant parameters $\{h_{rj}, s'\}$ according to the protocol and checks whether the equation $wP = P_{\text{pub}} + h_{rj} \cdot \bar{Y}_i + s' \cdot E + X_j$ holds. If it holds, **Game₂** obtains the session key k_{ji} with

acceptance; otherwise, **Game₂** outputs a failure and terminates the process.

In **Game₂**, we exclude the events of hash collision and random number collision, the probability of which is no more than $\frac{q_h^2}{2p} + \frac{(q_s + q_e)^2}{2p}$ according to the birthday paradox. Obviously, **Game₂** and **Game₁** are indistinguishable if the collision does not happen. Thus,

$$|\Pr[\text{Succ}_2(\mathcal{A})] - \Pr[\text{Succ}_1(\mathcal{A})]| \leq \frac{q_h^2}{2p} + \frac{(q_s + q_e)^2}{2p}.$$

Game₃: In this game, the event that $\mathcal{A}$ may obtain the parameter d to calculate the session key is excluded. According to the proposed protocol, $\mathcal{A}$ may utilize two approaches to obtain d .

- (1) Since the parameter D is a public parameter and $D = dP$, $\mathcal{A}$ may obtain d from D , and the probability is no more than ϵ_{ECDL} according to the ECDL assumption.
- (2) $\mathcal{A}$ may directly guess the parameter d , and the probability is no more than $\frac{q_d}{p}$.

The difference between **Game₃** and **Game₂** is whether $\mathcal{A}$ calculates the session key through the parameter d . If this event does not occur, **Game₃** and **Game₂** are indistinguishable. Thus,

$$|\Pr[\text{Succ}_3(\mathcal{A})] - \Pr[\text{Succ}_2(\mathcal{A})]| \leq \epsilon_{\text{ECDL}} + \frac{q_d}{p}.$$

Game₄: In this game, the event that $\mathcal{A}$ may obtain the parameter e to calculate the session key is further excluded. According to the proposed protocol, $\mathcal{A}$ may utilize two approaches to obtain e .

- (1). Since the parameter E is a public parameter and $E = eP$, $\mathcal{A}$ may obtain e from E , and the probability is no more than ϵ_{ECDL} according to the ECDL assumption.
- (2). $\mathcal{A}$ may directly guess the parameter e , and the probability is no more than $\frac{q_e}{p}$.

Obviously, **Game₄** and **Game₃** are indistinguishable if the event that $\mathcal{A}$ calculates the session key through parameter e does not occur. Thus,

$$|\Pr[\text{Succ}_4(\mathcal{A})] - \Pr[\text{Succ}_3(\mathcal{A})]| \leq \epsilon_{\text{ECDL}} + \frac{q_e}{p}.$$

Game₅: In this game, the event that $\mathcal{A}$ may obtain the parameter deP to calculate the session key is excluded. Since the previous games **Game₃** and **Game₄** have excluded the case that $\mathcal{A}$ obtains d and e , in this game, $\mathcal{A}$ can only obtain deP through the public parameters $D = dP$ and $E = eP$. The probability of this event is no more than ϵ_{ECDH} according to the ECCDH assumption.

The difference between **Game₅** and **Game₄** is whether $\mathcal{A}$ calculates the session key through the parameter deP . If this event does not occur, **Game₅** and **Game₄** are indistinguishable. Thus,

$$|\Pr[\text{Succ}_5(\mathcal{A})] - \Pr[\text{Succ}_4(\mathcal{A})]| \leq \epsilon_{\text{ECDH}}.$$

Game₆: In this game, the event that $\mathcal{A}$ may calculate the session key k by k' is excluded, where k' is the session key in the previous or subsequent session. From the protocol, it is easy to know that the current session key $k = H_2(\text{SN}_j, deP)$ has nothing to do with k' . Thus,

$$\Pr[\text{Succ}_6(\mathcal{A})] = \Pr[\text{Succ}_5(\mathcal{A})].$$

Based on the previous games, **Game₆** has already excluded the parameters $\mathcal{A}$ may utilize to calculate the session key. Therefore, in **Game₆**, $\mathcal{A}$ can only win the coin game by guessing, and we have

$$\Pr[\text{Succ}_6(\mathcal{A})] = \frac{1}{2}.$$

Additionally, according to the *triangular inequality*, we know

$$|\Pr[\text{Succ}_i(\mathcal{A})] - \Pr[\text{Succ}_k(\mathcal{A})]| \leq |\Pr[\text{Succ}_i(\mathcal{A})] - \Pr[\text{Succ}_j(\mathcal{A})]| + |\Pr[\text{Succ}_j(\mathcal{A})] - \Pr[\text{Succ}_k(\mathcal{A})]|.$$

From the above analysis, we have

$$\text{Adv}_{\mathcal{D}}^{\text{SK}}(\mathcal{A}) \leq \frac{q_h^2 + (q_s + q_e)^2 + 4q_s}{p} + 2\epsilon_{\text{puf}} + 4\epsilon_{\text{ECDL}} + 2\epsilon_{\text{ECCDH}}.$$

6.2. Informal security analysis

In addition to the above formal security analysis, this paper also utilizes a non-mathematical method to prove the protocol's security features.

- (1) *Mutual Authentication.* According to the protocol, the service node SN_j verifies the legitimacy of terminal device TD_i by judging whether the equation $wP = P_{\text{pub}} + hr_j \cdot \bar{Y}_i + s \cdot E + X_j$ holds. Since $w = \bar{y}_i + e \cdot s$ and $\bar{y}_i = (u + v)^{-1} \cdot (u \cdot y_{i1} + v \cdot y_{i2})$, only the legal devices possessing parameters (y_{i1}, Y_{i1}) and (y_{i2}, Y_{i2}) can calculate the right $(\bar{y}_i, \bar{Y}_i)$ to make the above equation hold. Meanwhile, TD_i verifies the legitimacy of SN_j by judging whether the equation $\bar{h} = h$ holds. Since the ECDL and ECCDH problems are intractable and the parameter $\bar{h} = H_3(SN_j, D, eX_j) = H_3(SN_j, D, ex_jP) = H_3(SN_j, D, x_jE)$, in addition to TD_i , only SN_j possessing the secret x_j can correctly calculate the parameter $\bar{h}$.
- (2) *Secure Session Key.* In the protocol, the terminal device TD_i calculates the session key by performing $k_{ij} = H_2(SN_j, eD)$. And the service node SN_j calculates the session key as $k_{ji} = H_2(SN_j, dE)$. Due to $k_{ji} = H_2(SN_j, eD) = H_2(SN_j, edP) = H_2(SN_j, dE) = k_{ij}$, we know that TD_i and SN_j could establish the shared session key during the anonymous authentication process. Besides, according to the ECDL assumption and the ECCDH assumption, the adversary cannot obtain the parameter edP to calculate the session key.
- (3) *Strong Anonymity.* According to the proposed protocol, the terminal device TD_i sends the message $\langle c_j, p_j, ts_j, E \rangle$ to SN_j as the anonymous authentication request, where $E = eP$ is calculated from the random value e and the parameters (c_j, p_j, ts_j) are identical for any terminal devices requesting service from SN_j over the same period. Therefore, SN_j cannot obtain the identity of TD_i from this request message. Besides, TD_i generates a new temporary public/private key pair $(\bar{y}_i, \bar{Y}_i)$ by performing $\bar{y}_i = (u + v)^{-1} \cdot (u \cdot y_{i1} + v \cdot y_{i2})$ and $\bar{Y}_i = (u + v)^{-1} \cdot (uY_{i1} + vY_{i2})$ in each communication, and sends the message $\langle w, \bar{Y}_i \rangle$ to SN_j as authentication parameters, where $w = \bar{y}_i + e \cdot s$ and $s = H_4(SN_j, D, \bar{Y}_i, eX_j)$. From the related parameters, SN_j can only verify the legitimacy of TD_i but cannot obtain its real identity information. Additionally, due to the randomness of u and v , the management server MS also cannot infer the real identity of TD_i even if MS possesses the parameters (y_{i1}, Y_{i1}) and (y_{i2}, Y_{i2}) . If it is necessary for audit or investigation, MS may require TD_i to submit the parameters u and v in this communication. Based on this, MS can find (Y_{i1}, Y_{i2}) that satisfies the condition $\bar{Y}_i = (u + v)^{-1} \cdot (uY_{i1} + vY_{i2})$ by traversing the parameter list, and then the identity information of the terminal device is obtained.
- (4) *Unlinkability.* In the protocol, each time the terminal device TD_i requests service from the service node SN_j , TD_i generates its temporary public/private key pair $(\bar{y}_i, \bar{Y}_i)$ by performing $\bar{y}_i = (u + v)^{-1} \cdot (u \cdot y_{i1} + v \cdot y_{i2})$ and $\bar{Y}_i = (u + v)^{-1} \cdot (uY_{i1} + vY_{i2})$, where u and v are random values selected by TD_i . Since the parameters u and v are generated independently in each communication and have nothing to do with the previous or subsequent communication, SN_j cannot link the terminal device by collecting the parameters in each round of communication.

- (5) *Resistance to Eavesdropping Attacks.* According to **Theorem 1**, $\mathcal{A}$ cannot obtain any useful information from the messages transmitted in the public channel. That is, eavesdropping cannot increase the advantage of $\mathcal{A}$ in attacking the protocol.
- (6) *Resistance to Replay Attacks.* Each time the process of anonymous authentication is executed, the terminal device TD_i selects new random parameters u and v to generate its temporary public/private key pair. Meanwhile, TD_i and SN_j respectively select random parameters e and d to ensure the freshness of the transmitted messages.
- (7) *Resistance to Data Tampering Attacks.* The proposed protocol uses hash operations with secret parameters to ensure the integrity of the messages. Taking $h = H_3(SN_j, D, x_jE)$ as an example, due to the confidentiality of parameter x_j , the adversary cannot calculate the correct hash value. Meanwhile, the malicious tampering of messages by the adversary can be easily recognized by the protocol participants.
- (8) *Resistance to Physical Attacks.* By performing physical attacks, the adversary can obtain $\langle px_j, X_j, ts_j \rangle$ that is stored in the target device SN_j . The private key of SN_j is calculated as $x_j = r_j \oplus px_j$, where $r_j = \text{Rep}(\bar{w}_j, p_j)$ and $\bar{w}_j = \text{PUF}_j(c_j)$. Since any attempt to tamper the PUF will make it useless [14], the adversary cannot obtain r_j to calculate the private key x_j of SN_j . For the same reason, the adversary cannot obtain any secret parameters by performing physical attacks on the terminal device TD_i .
- (9) *Resistance to Known-Key Attacks.* The unique session key in each session of the protocol is calculated as $k = H_2(SN_j, edP)$, where e and d are random values. Hence, even if the adversary has obtained the key k' in the previous or subsequent session, he/she still has to calculate the current session key k from scratch.
- (10) *Resistance to Modeling Attacks.* In the protocol, the management server MS stores the hash values of parameters related to PUF, such as $hr_j = H_1(r_j \| c_j \| x_j \| ts_j)$, instead of storing the raw CRPs. Besides, the terminal devices and service nodes also do not need to store the CRPs of PUF. Due to the proposed protocol not exposing any CRPs of PUF, it can effectively resist the PUF modeling attacks.
- (11) *Perfect Forward Secrecy.* According to the protocol, the session key $k = H_2(SN_j, eD) = H_2(SN_j, dE) = H_2(SN_j, edP)$. Even if the long-term keys of TD_i and SN_j , $\langle (y_{i1}, Y_{i1}), (y_{i2}, Y_{i2}), (x_j, X_j) \rangle$, are cracked by the adversary, it is still hard to calculate the parameter edP according to the ECCDH assumption.

6.3. Potential vulnerabilities and countermeasures

Similar to existing PUF-based authentication protocols, such as protocols [17,18], the security of the proposed protocol depends on the unpredictability and uniqueness of PUF. In recent years, numerous studies on attacks against PUF have been conducted, among which machine learning attacks [30] and side-channel attacks [31] have received more attention. In addition, we analyzed the impact of insider threats and advanced persistent threats (APT), which may challenge the IoT ecosystem, on the protocol.

- (1) *Machine learning attacks.* The prerequisite for implementing machine learning attacks on PUF is that adversaries have collected a subset of CRPs of the PUF. The proposed protocol uses the hash values of parameters related to PUF instead of using the raw CRPs, so it is hard for adversaries to obtain sufficient CRPs of the PUF to implement machine learning attacks. Besides, the PUF design method that resists machine learning attacks, such as [32], can be selected in practical applications.
- (2) *Side-channel attacks.* Although adversaries have difficulty directly accessing raw CRPs, they may obtain valuable information through side-channel attacks, thereby threatening the security of

the protocol. Fortunately, the proposed protocol uses PUF as a security primitive and such attacks can be prevented by selecting side-channel attacks resilient PUF, such as [31,33].

- (3) *Insider threats.* Among various IoT security threats, insider threats are dangerous for the reason that insider attack can be launched by accessing the system with less efforts [34]. In some protocols, malicious insiders may be able to easily obtain security parameters as authentication factors, such as the password, to launch attacks. However, the proposed protocol relies on PUF for authentication, which is bound to specific device by exploiting the unique hardware features. According to the unpredictability and uniqueness of PUF, it is hard for both external and internal adversaries to obtain the parameters used for authentication.
- (4) *Advanced persistent threats.* Different from the attacks done by a regular hacker, APT attacks are achieved by a group of advance attackers who are highly persistent to achieve the goal of accessing targets or data [35]. And APT attacks are typically implemented by collecting information and exploiting vulnerabilities. As for the proposed protocol, through the security analysis, it is known that adversaries cannot obtain any valuable information from the execution process. Therefore, for the protocol itself, APT attacks cannot compromise its security. But for the entire IoT ecosystem, it is necessary to take reasonable solutions, as given in [35], to defend against APT attacks.

7. Experimental validation and performance analysis

In this section, we present the experimental results, and then the proposed protocol is compared with relevant protocols from the aspects of security features, protocol rounds, communication cost, and computation cost.

7.1. Experimental validation

The Raspberry Pi 4B development boards with 8GB RAM are adopted to carry out the experiment. The 5-4 DAPUF [36] is selected and implemented on Xilinx Zynq-7010 (XC7Z010) FPGA platform using Vivado 2018.3 development environment. And the fuzzy extractor is designed by means of BCH code. The PUF module is connected to the Raspberry Pi board through serial cable. Besides, the development boards acting as terminal device and service node communicate with each other through the WiFi channel.

In the experiment, we use Python to implement the functions of terminal device and service node. The curve secp256k1 [37], which achieves the 128-bit security level, is used to perform cryptographic operations on elliptic curves. As for hash functions, the SHA256 algorithm is adopted. In addition, the typical length of input challenge of 5-4 DAPUF is 64 bits, and the length of corresponding output response is 4 bits. To facilitate the implementation, the length of challenges and responses of PUF module is set to 128 bits with the help of linear feedback shift register (LFSR). That is to say, under the action of LFSR, the 128-bit original challenge is used as a seed to generate 32 different 64-bit challenges that are fed to the DAPUF. Besides, the registration parameters are stored in advance in the devices.

On the basis of the above experimental environment and configuration, the anonymous authentication process was tested for 10 times, and Fig. 6 shows the time duration of each execution.

From the experimental results, we can see that the anonymous authentication between terminal device and service node can be completed within 2.2 s. It is worth noting that the time duration includes not only the time for performing the cryptographic operations and communication, but also the time spent on serial communication, which can be saved if PUF is integrated into the chip of device controller. In addition, the time cost of performing complex cryptographic operations is also closely related to the programming language used. To prove this point, we test the execution time of common complex cryptographic

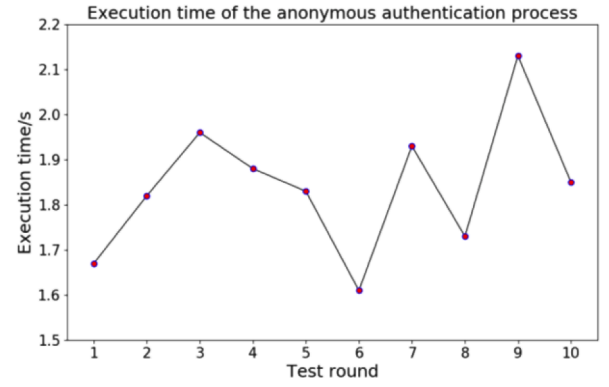


Fig. 6. Execution time of the anonymous authentication process.

operations using Python and C++, respectively, based on the MIRACL cryptography library [38] on the Raspberry Pi 4B development board. The operations of point addition and the point multiplication are tested on the elliptic curve secp256k1 [37]. And Optimal Ate Pairing [39] over the pairing-friendly curve BLS12-381 [40] is adopted to perform bilinear pairing operation. The average running time of common complex cryptographic operations is reported in Table 3.

In addition to the experimental verification mentioned above, the feasibility of the proposed protocol on low-power IoT devices is further discussed by taking the Philips HiPersmart card and TI's CC2538 hardware platform for example. According to the proposed protocol, we know that the cryptographic operations adopted mainly include the general hash, the modular multiplication, the point addition and point multiplication over the elliptic curve, excluding the bilinear pairing operations with high computational complexity. The running time of above cryptographic operations on the Philips HiPersmart card and TI's CC2538 hardware platform are respectively presented in [7,23]. In other words, the cryptographic operations used in the proposed protocol can be implemented on the Philips HiPersmart card and TI's CC2538 hardware platform. Therefore, as long as necessary modules, such as the communication components and memory, meet the requirements, the proposed protocol is feasible to run on low-power IoT devices.

7.2. Security features

Table 4 shows the comparison results of security features. It can be seen that the proposed protocol meets all important security features, while other protocols face some security threats. The protocols based on the traditional cryptography [4,7,8] face the threat of physical attacks (S8), which can be avoided by the PUF-based protocols [17–22]. But the protocols [17–21] directly utilize the raw CRPs of PUF for authentication, making them susceptible to modeling attacks (S10). The protocol [22] is free from modeling attacks, but this protocol does not provide anonymity guarantees (S3). Besides, the PUF-based anonymous authentication protocols [17–21] all provide a weak anonymity mechanism, allowing the verifier to obtain the real identity of IoT devices, which may not meet the needs of some scenarios with high security requirements.

7.3. Communication cost

To effectively analyze the communication cost, the length of device identity is defined as 32 bits, the length of PUF's parameters is defined as 128 bits, and the length of the hash value, the random number, the private key are all defined as 256 bits. Besides, the length of a point $P(P_x, P_y)$ on the elliptic curve is defined as 512 bits, where P_x denotes the x coordinate and P_y denotes the y coordinate.

Table 5 shows the comparison of communication costs between the proposed protocol and relevant protocols. From Table 5, it can be seen

Table 3

Average running time (Millisecond) of complex cryptographic operations.

Operations	Modular Multiplication	Modular Inversion	Point Addition	Point Multiplication	Bilinear Pairing	Hash	Symmetric encryption & decryption
Time Cost (Python)	0.006	0.211	0.284	110.277	2080.992	0.007	0.126
Time Cost (C++)	0.003	0.010	0.012	2.352	24.436	0.003	0.009

Table 4

Comparison of security features.

Protocols	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11
Gope et al. (2019) [17]	✓	✓	weak	–	✓	✓	✓	✓	✓	×	✓
Suganthi et al. (2019) [19]	✓	✓	weak	–	✓	✓	✓	✓	✓	×	✓
Banerjee et al. (2019) [20]	✓	✓	weak	–	✓	✓	✓	✓	✓	×	✓
Chatterjee et al. (2019) [22]	✓	×	×	×	✓	✓	✓	✓	✓	✓	✓
Tedeschi et al. (2020) [4]	✓	✓	×	×	✓	✓	✓	×	✓	–	✓
Srinivas et al. (2021) [7]	✓	✓	weak	–	✓	✓	✓	×	✓	–	✓
Liu et al. (2020) [21]	✓	✓	weak	–	✓	✓	✓	✓	✓	×	✓
Ding et al. (2022) [8]	✓	✓	strong	✓	✓	✓	✓	×	✓	–	✓
Subramani et al. (2022) [18]	✓	✓	weak	–	✓	✓	✓	✓	✓	×	✓
Proposed Protocol	✓	✓	strong	✓	✓	✓	✓	✓	✓	✓	✓

Note: S1: Mutual Authentication, S2: Secure Session Key, S3: Anonymity, S4: Unlinkability, S5: Resistance to Eavesdropping Attacks, S6: Resistance to Replay Attacks, S7: Resistance to Data Tampering Attacks, S8: Resistance to Physical Attacks, S9: Resistance to Known-Key Attacks, S10: Resistance to Modeling Attacks, S11: Perfect Forward Secrecy.

Table 5

Comparison of protocol round and communication cost.

Protocols	Protocol Round	Communication Cost (bits)			
		Initiator	Server/Gateway	Verifier	Total Cost
Gope et al. (2019) [17]	6	704	2112	384	3200
Suganthi et al. (2019) [19]	5	896	1280	608	2784
Banerjee et al. (2019) [20]	3	1280	896	1024	3200
Chatterjee et al. (2019) [22]	7	2112	5440	2048	9600
Liu et al. (2020) [21]	4	576	1856	832	3264
Subramani et al. (2022) [18]	6	1856	1600	1856	5312
Proposed Protocol	3	1568	0	768	2336

Note: The protocol round and communication cost in Table V refer to these in the authentication phase of the protocols. And the communication cost is measured by the length of messages the device or server sends.

that the proposed protocol outperforms relevant protocols in terms of communication cost. Besides, the protocol rounds of the proposed protocol and protocol [20] are less than other protocols.

7.4. Computation cost

To facilitate analysis and comparison, the computation cost is measured by the execution times of complex operations. Let t_p , t_{fe} , t_h , t_{mph} , t_{enc} , t_{ea} , t_{em} , and t_{bp} denote the time cost for the PUF operation, the fuzzy extractor operation, the general hash, the map-to-point hash, the symmetric encryption/decryption, the point addition over the elliptic curve, the point multiplication over the elliptic curve, and the bilinear pairing operation, respectively. Some lightweight operations, such as XOR, are omitted in the analysis.

Table 6 presents the comparison of the proposed protocol and relevant protocols in terms of computation cost. According to the time cost of complex operations described in [21,41], we know: (1) the computation cost of the proposed protocol is less than the protocol [22], which uses the bilinear pairing operations with large computational complexity; (2) the proposed protocol adopting the operations of point

Table 6

Comparison of computation cost.

Protocols	Computation Cost		
	Initiator	Server/Gateway	Verifier
Gope et al. (2019) [17]	$3t_p + 6t_h$	$9t_h$	$2t_p + 4t_h$
Suganthi et al. (2019) [19]	$t_p + t_{fe} + 2t_h + t_{enc}$	$5t_h + t_{enc}$	$3t_h$
Banerjee et al. (2019) [20]	$t_p + t_{fe} + 17t_h$	$8t_h$	$t_p + t_{fe} + 6t_h$
Chatterjee et al. (2019) [22]	$t_p + t_{fe} + 2t_h + 7t_{mph} + 2t_{ea} + 2t_{em} + t_{bp}$	$10t_h + 6t_{mph} + 12t_{ea} + 4t_{em} + 2t_{bp}$	$t_p + t_{fe} + 2t_h + 7t_{mph} + 2t_{ea} + 2t_{em} + t_{bp}$
Liu et al. (2020) [21]	$t_{fe} + 7t_h + 2t_{enc}$	$t_{fe} + 11t_h + 5t_{enc}$	$2t_p + 3t_{fe} + 6t_h + 2t_{enc}$
Subramani et al. (2022) [18]	$2t_p + 2t_f + 6t_h$	$5t_h + 2t_f$	$2t_p + 2t_f + 6t_h$
Proposed Protocol	$t_p + t_{fe} + 3t_h + t_{ea} + 6t_{em}$	0	$t_p + t_{fe} + 4t_h + 3t_{ea} + 6t_{em}$

Note: The computation cost in Table VI refers to that in the authentication phase of the protocols.

addition and point multiplication over the elliptic curve is inferior to the protocols [17–21] which mainly use the hash or symmetric encryption/decryption operations. However, it is worth noting that the proposed protocol realized the direct authentication between initiator and verifier without needing the real-time participation of the third-party server, which greatly mitigates the burden on the server and effectively reduces the risk of a single point of failure, especially in IoT with a large number of terminal devices.

8. Conclusion

By combining PUF with certificateless public key cryptography on the elliptic curve, this paper proposes a novel anonymous authentication protocol that effectively realizes the end-to-end direct mutual authentication and session key agreement while maintaining strong anonymity. The security analysis indicates that the protocol provides perfect forward secrecy while resisting various security threats including physical attacks and modeling attacks. Without needing the real-time participation of the third-party server, the proposed protocol outperforms related

protocols in terms of protocol rounds and communication costs, which is more practical and flexible for anonymous authentication for IoT devices.

CRedit authorship contribution statement

Sensen Li: Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Yicai Huang:** Writing – review & editing, Methodology, Conceptualization. **Bin Yu:** Validation, Supervision, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- [1] Juniper Research whitepaper, IoT ~ The internet of transformation 2020. <https://www.juniperresearch.com/whitepapers/iot-the-internet-of-transformation-2020>. (Accessed 5 September 2022).
- [2] M. Frustaci, P. Pace, G. Aloï, G. Fortino, Evaluating critical security issues of the IoT world: present and future challenges, *IEEE Internet Things J.* 5 (4) (2018) 2483–2495, <https://doi.org/10.1109/JIOT.2017.2767291>.
- [3] B. Hammi, A. Payad, R. Khatoun, S. Zeadally, Y. Begriche, A lightweight ECC-based authentication scheme for Internet of Things (IoT), *IEEE Syst. J.* 14 (3) (2020) 3440–3450, <https://doi.org/10.1109/JSYST.2020.2970167>.
- [4] P. Tedeschi, S. Sciancalepore, A. Eliyan, R. Di Pietro, LiKe: lightweight certificateless key agreement for secure IoT communications, *IEEE Internet Things J.* 7 (1) (2020) 621–638, <https://doi.org/10.1109/JIOT.2019.2953549>.
- [5] S. Malani, J. Srinivas, A.K. Das, K. Srinathan, M. Jo, Certificate-based anonymous device access control scheme for IoT environment, *IEEE Internet Things J.* 6 (6) (2019) 9762–9773, <https://doi.org/10.1109/JIOT.2019.2931372>.
- [6] C. Shouqi, L. Wanrong, C. Liling, H. Xin, J. Zhiyong, An improved authentication protocol using smart cards for the Internet of Things, *IEEE Access* 7 (2019) 157284–157292, <https://doi.org/10.1109/ACCESS.2019.2949649>.
- [7] J. Srinivas, A.K. Das, X. Li, M.K. Khan, M. Jo, Designing anonymous signature-based authenticated key exchange scheme for internet of things-enabled smart grid systems, *IEEE Trans. Ind. Inform.* 17 (7) (2021) 4425–4436, <https://doi.org/10.1109/TII.2020.3011849>.
- [8] X. Ding, X. Wang, Y. Xie, F. Li, A lightweight anonymous authentication protocol for resource-constrained devices in Internet of Things, *IEEE Internet Things J.* 9 (3) (2022) 1818–1829, <https://doi.org/10.1109/JIOT.2021.3088641>.
- [9] X. Li, J. Niu, S. Kumari, F. Wu, A.K. Sangaiah, K.K.R. Choo, A three-factor anonymous authentication scheme for wireless sensor networks in Internet of Things environments, *J. Netw. Comput. Appl.* 103 (2018) 194–204.
- [10] S. Banerjee, et al., A provably secure and lightweight anonymous user authenticated session key exchange scheme for Internet of Things deployment, *IEEE Internet Things J.* 6 (5) (2019) 8739–8752, <https://doi.org/10.1109/JIOT.2019.2923373>.
- [11] C. Marchand, et al., Implementation and characterization of a physical unclonable function for IoT: a case study with the TERO-PUF, *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 37 (1) (2018) 97–109, <https://doi.org/10.1109/TCAD.2017.2702607>.
- [12] U. Chatterjee, D. Mukhopadhyay, R.S. Chakraborty, 3PAA: a private PUF protocol for anonymous authentication, *IEEE Trans. Inf. Forensics Secur.* 16 (2021) 756–769, <https://doi.org/10.1109/TIFS.2020.3021917>.
- [13] R. Pappu, B. Recht, J. Taylor, N. Gershenfeld, Physical one-way functions, *Science* 297 (5589) (2002) 2026–2030, <https://doi.org/10.1126/science.1074376>.
- [14] M.N. Aman, M.H. Basheer, B. Sikdar, Data provenance for IoT with light weight authentication and privacy preservation, *IEEE Internet Things J.* 6 (6) (2019) 10441–10457, <https://doi.org/10.1109/JIOT.2019.2939286>.
- [15] P. Gope, B. Sikdar, Lightweight and privacy-preserving two-factor authentication scheme for IoT Devices, *IEEE Internet Things J.* 6 (1) (2019) 580–589, <https://doi.org/10.1109/JIOT.2018.2846299>.
- [16] M.A. Qureshi, A. Munir, PUF-IPA: a PUF-based identity preserving protocol for Internet of Things authentication, in: *Proceedings of the IEEE 17th Annual Consumer Communications & Networking Conference (CCNC)*, Las Vegas, NV, USA, 2020, pp. 1–7, <https://doi.org/10.1109/CCNC46108.2020.9045264>.
- [17] P. Gope, A.K. Das, N. Kumar, Y. Cheng, Lightweight and physically secure anonymous mutual authentication protocol for real-time data access in industrial wireless sensor networks, *IEEE Trans. Ind. Inform.* 15 (9) (2019) 4957–4968, <https://doi.org/10.1109/TII.2019.2895030>.
- [18] J. Subramani, A. Maria, A.S. Rajasekaran, F. Al-Turjman, Lightweight privacy and confidentiality preserving anonymous authentication scheme for WBANs, *IEEE Trans. Ind. Inform.* 18 (5) (2022) 3484–3491, <https://doi.org/10.1109/TII.2021.3097759>.
- [19] S.D. Suganthi, R. Anitha, V. Sureshkumar, End to end light weight mutual authentication scheme in IoT-based healthcare environment, *J. Reliab. Intell. Environ.* 6 (1) (2019) 3–11, <https://doi.org/10.1007/s40860-019-00079-w>.
- [20] S. Banerjee, V. Odelu, A.K. Das, S. Chattopadhyay, J.J.P.C. Rodrigues, Y. Park, Physically secure lightweight anonymous user authentication protocol for Internet of Things using physically unclonable functions, *IEEE Access* 7 (2019) 85627–85644, <https://doi.org/10.1109/ACCESS.2019.2926578>.
- [21] Z. Liu, C. Guo, B. Wang, A physically secure, lightweight three-factor and anonymous user authentication protocol for IoT, *IEEE Access* 8 (2020) 195914–195928, <https://doi.org/10.1109/ACCESS.2020.3034219>.
- [22] U. Chatterjee, et al., Building PUF based authentication and key exchange protocol for IoT without explicit CRPs in verifier database, *IEEE Trans. Dependable Secure Comput.* 16 (3) (2019) 424–437, <https://doi.org/10.1109/TDSC.2018.2832201>.
- [23] S. Li, T. Zhang, B. Yu, K. He, A provably secure and practical PUF-based end-to-end mutual authentication and key exchange protocol for IoT, *IEEE Sens. J.* 21 (4) (2021) 5487–5501.
- [24] Y. Zheng, W. Liu, C. Gu, C.H. Chang, PUF-based mutual authentication and key exchange protocol for peer-to-peer IoT applications, *IEEE Trans. Dependable Secure Comput.* (2022), <https://doi.org/10.1109/TDSC.2022.3193570>.
- [25] U. Rührmair, J. Sölter, PUF modeling attacks: an introduction and overview, in: *Proceedings of the 2014 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Dresden, Germany, 2014, pp. 1–6, <https://doi.org/10.7873/DATE.2014.361>.
- [26] Y. Dodis, R. Ostrovsky, L. Reyzin, A. Smith, Fuzzy extractors: how to generate strong keys from biometrics and other noisy data, *SIAM J. Comput.* 38 (1) (2008) 97–139, <https://doi.org/10.1137/060651380>.
- [27] D. Dolev, A. Yao, On the security of public key protocols, *IEEE Trans. Inf. Theory* 29 (2) (1983) 198–208, <https://doi.org/10.1109/TIT.1983.1056650>.
- [28] M. Bellare, D. Pointcheval, P. Rogaway, Authenticated key exchange secure against dictionary, in: *Proceedings of the International Conference on the Theory & Applications of Cryptographic Techniques (EUROCRYPT)*, Bruges, Belgium, 2000, pp. 139–155, https://doi.org/10.1007/3-540-45539-6_11.
- [29] V. Shoup, Sequences of games: A tool for taming complexity in security proofs, *IACR Cryptology ePrint Archive*, 2004, <https://eprint.iacr.org/2004/332>. (Accessed 13 September 2023).
- [30] J. Delvaux, Machine-learning attacks on PolyPUFs, OB-PUFs, RPUFs, LHS-PUFs, and PUF-FSMs, *IEEE Trans. Inf. Forensics Secur.* 14 (8) (2019) 2043–2058, <https://doi.org/10.1109/TIFS.2019.2891223>.
- [31] A. Aghaie, A. Moradi, TI-PUF: toward side-channel resistant physical unclonable functions, *IEEE Trans. Inf. Forensics Secur.* 15 (2020) 3470–3481, <https://doi.org/10.1109/TIFS.2020.2986887>.
- [32] R. Ali, H. Ma, Z. Hou, D. Zhang, E. Deng, Y. Wang, A reconfigurable arbiter MPUF with high resistance against machine learning attack, *IEEE Trans. Magn.* 57 (10) (2021) 1–7, <https://doi.org/10.1109/TMAG.2021.3102838>. Art no. 3402007.
- [33] L. Tebelmann, J.L. Danger, M. Pehl, Interleaved challenge loop PUF: a highly side-channel protected oscillator-based PUF, *IEEE Trans. Circuits Syst. I Regul. Pap.* 69 (12) (2022) 5121–5134, <https://doi.org/10.1109/TCSI.2022.3208325>.
- [34] S. Rajamanickam, S. Vollala, R. Amin, N. RamaSubramanian, Insider attack protection: lightweight password-based authentication techniques using ECC, *IEEE Syst. J.* 14 (2) (2020) 1972–1983, <https://doi.org/10.1109/JSYST.2019.2933464>.
- [35] A. Alshamrani, S. Myneni, A. Chowdhary, D. Huang, A survey on advanced persistent threats: techniques, solutions, challenges, and research opportunities, *IEEE Commun. Surv. Tutor.* 21 (2) (2019) 1851–1877, <https://doi.org/10.1109/COMST.2019.2891891>.
- [36] U. Chatterjee, D.P. Sahoo, D. Mukhopadhyay, R.S. Chakraborty, Trustworthy proofs for sensor data using FPGA based physically unclonable functions, in: *Proceedings of the Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Dresden, Germany, 2018, pp. 1504–1507, <https://doi.org/10.23919/DATE.2018.8342252>.
- [37] Brown DRL, SEC 2: Recommended elliptic curve domain parameters. Standards for Efficient Cryptography, 2010, <https://www.secg.org/sec2-v2.pdf>. (Accessed 28 April 2024).
- [38] MIRACL, Miracl cryptographic SDK: Multiprecision integer and rational arithmetic cryptographic library. <https://github.com/miracl>. (Accessed 28 December 2023).
- [39] F. Vercauteren, Optimal pairings, *IEEE Trans. Inf. Theory* 56 (1) (2010) 455–461.
- [40] S. Bowe, BLS12-381: New zk-SNARK elliptic curve construction. <https://electricoin.co/blog/new-snark-curve/>. (Accessed 28 December 2023).
- [41] L. Wu, J. Wang, K.K.R. Choo, D. He, Secure key agreement and key protection for mobile device user authentication, *IEEE Trans. Inf. Forensics Secur.* 14 (2) (2019) 319–330, <https://doi.org/10.1109/TIFS.2018.2850299>.



Sensen Li received the B.S. degree and M.S. degree from the Zhengzhou Information Science and Technology Institute in 2015 and 2018, respectively. He is currently pursuing the Ph.D. degree with the Department of Computer Science, Zhengzhou Information Science and Technology Institute, China. His research interests include Internet of Things security, wireless network, and secure embedded systems.



Bin Yu received the B.S. degree from the Department of Electronic Engineering, University of Shanghai Jiaotong, in 1986, the M.S. degree from the Department of Automatic Engineering, South China University of Technology, in 1991, and the Ph.D. degree in 1999. From 1997 to 1999, he was a Research Assistant with The Hong Kong University of Science and Technology. From 2003 to 2004, he was a Vice Professor with the University of Waterloo, ON, Canada. He is currently a Professor with the Department of Computer Science, Zhengzhou Information Science and Technology Institute, China. His research interests include the design and analysis of algorithms, visual secret sharing, and network security.



Yicai Huang received the B.S. degree and M.S. degree from the Zhengzhou Information Science and Technology Institute in 2007 and 2010, respectively. He is currently pursuing the Ph.D. degree with the Department of Computer Science, Zhengzhou Information Science and Technology Institute, China. His research interests include Wireless Communication Security, Cryptographic Cloud Storage and Searchable Encryption.